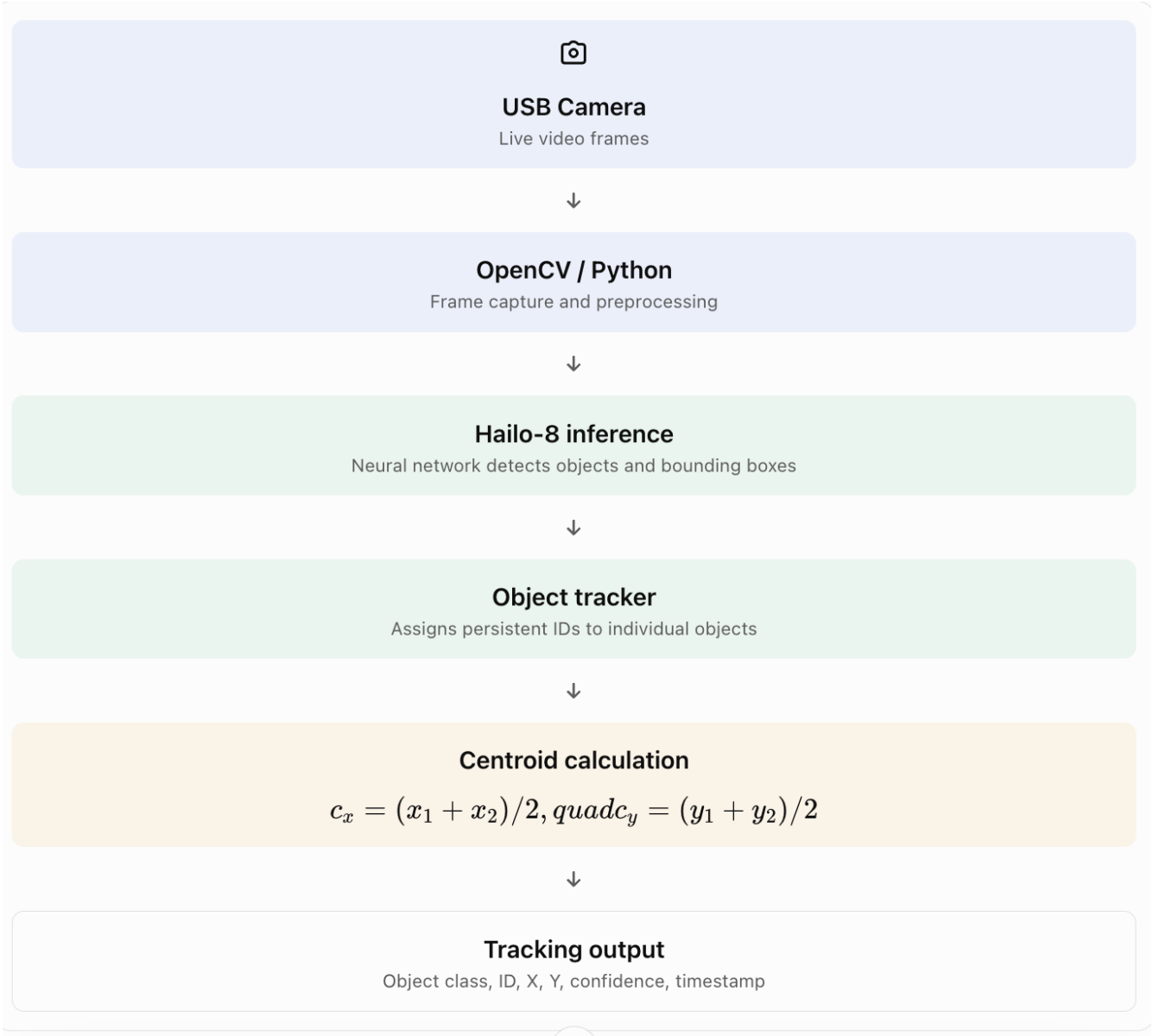
I have a Raspberry Pi 5 with a 13TOPS HAT+ running Hailo-8. I want to use Python and a USB camera to track multiple instances of three object types. I want to derive each object's centroid on the screen.

Your Raspberry Pi 5 with the Hailo-8 13 TOPS AI HAT+ is well suited to this application. I would build this using Python, Hailo object detection, and OpenCV.
The architecture I'd recommend is:

1. Capture video from a USB camera.
2. Run an object-detection neural network on the Hailo-8.
3. Detect multiple instances of three specified object classes.
4. Track individual objects between frames.
5. Calculate each object's centroid in camera-image coordinates.
6. Publish or display the coordinates in real time.

An important distinction: object detection identifies objects in individual frames; object tracking maintains an object's identity across frames. You'll probably want both.

# 1. Recommended software architecture




# 2. Object detection model

I recommend starting with YOLO.

Some good candidates:

| Model | Purpose |
|---|---|
| YOLOv8n | Lightweight, good starting point |
| YOLOv11n | Newer lightweight alternative |
| YOLOv8s | More detection capacity, higher compute requirements |

You can train or fine-tune a model to recognize exactly your three object types.

The model will return bounding boxes:

- `x1, y1`: top-left corner

- `x2, y2`: bottom-right corner
- `confidence`: detection confidence
- `class_id`: object category

You can have any number of detections for each of your three classes.
The Hailo accelerator runs the neural network, rather than requiring the Raspberry Pi CPU to perform the full inference workload.

# 3. Object tracking

For multiple instances of the same object, I recommend ByteTrack.
For example, suppose the camera sees:

- 4 red objects
- 3 blue objects
- 2 yellow objects

The tracker assigns a unique ID to each detected object.

| Class | Tracking ID | Centroid X | Centroid Y |
|---|---|---|---|
| Red | 1 | 245 | 312 |
| Red | 2 | 512 | 187 |
| Red | 3 | 690 | 425 |
| Blue | 4 | 385 | 260 |
| Blue | 5 | 720 | 350 |
| Yellow | 6 | 410 | 510 |

ByteTrack is a good choice because it associates detections between frames, including some lower-confidence detections. Hailo's own Python examples support tracking with ByteTrack.

# 4. Centroid calculation

This is straightforward in Python.
Given a detection bounding box:

```
x1, y1, x2, y2 = bbox
cx = int((x1 + x2) / 2)
cy = int((y1 + y2) / 2)
```

I would create a data structure for each tracked object:

```
from dataclasses import dataclass
@dataclass
class TrackedObject:
    object_id: int
    class_id: int
    confidence: float
    cx: int
    cy: int
    x1: int
    y1: int
    x2: int
    y2: int
    timestamp: float
```

Your application could maintain a list of these objects for every frame.

For visualization, OpenCV can draw the bounding boxes, centroids, and tracking IDs:

```
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
cv2.circle(frame, (cx, cy), 5, color, -1)
cv2.putText(
    frame,
    f"ID: {object_id}",
    (x1, y1 - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.5, color,2
)
```

One important detail: make sure the bounding boxes are mapped back to the original camera resolution before calculating the final pixel coordinates.

# 5. Getting started with Hailo

Rather than writing the Hailo interface from scratch, I'd start with Hailo's existing Python application examples.

The repository includes:

- USB camera input.
- YOLO object detection.
- Object tracking.
- Detection visualization.
- Python application examples.

For example, its detection application can be started with:

```
python basic_pipelines/detection.py --input usb
```

The Hailo standalone object detection application also supports a tracking option:

```
./object_detection.py -n yolov8n.hef -i usb --track
```

See the Hailo object detection documentation:
https://github.com/hailo-ai/hailo-apps/tree/main/hailo_apps/python/standalone_apps/object_detection

# 6. A few design decisions before implementing your application

I'd like to establish four things so that I can build the actual Python application around your hardware.

1. What are the three object types?

Are they standard objects (such as balls, people, and tools), or are they custom objects that you need to train a model to recognize?

2. How many instances might appear simultaneously?

For example, 5, 10, 20, or more?

3. What is your camera resolution and frame rate?

For example, 1920 × 1080 at 30 FPS.

4. What do you want to do with the centroid coordinates?

- Display them on the video.
- Produce a continuously updated list of object IDs and coordinates.
- Publish coordinates through NetworkTables.
- Send coordinates to another Python application.

My recommendation is to build a modular application with separate camera, detector, tracker, and centroid modules. This will make it easier to optimize performance and change detection models later. Once these decisions are established, I can implement the complete Python application for your Pi 5 and Hailo-8.

I don't know what the objects will be yet, but I will want to train a custom model. And I don't know how many I'll need to track. I believe that the Hailo process scales the images to a specific size. Should the camera use that resolution? And, yes, I'll be using Network Tables.

Yes. I would design this as a customizable, multi-object vision system with Hailo-8 inference and NetworkTables output, rather than building it around any particular object type.
There are two important design decisions here:

1. The camera resolution does not necessarily need to match the neural network's input resolution.
2. The number of objects should be determined dynamically from the detections, not configured as a fixed number.

Both decisions will make your system much more flexible as your project develops.

## 1. Camera resolution versus neural network resolution

You're correct that Hailo models generally have a specific input resolution.
For example, a YOLO model might expect:

# 640 × 640

Neural network input
Fixed model dimensions

# 1920 × 1080

Camera input
Example USB camera resolution
The typical processing sequence is:

1. Capture a 1920 × 1080 camera frame.
2. Resize or letterbox it to 640 × 640.
3. Send the resulting image to Hailo.
4. Receive detection bounding boxes.
5. Transform the bounding boxes back into the original 1920 × 1080 coordinate system.
6. Calculate centroids using the original image coordinates.

My recommendation: Don't automatically set the camera to the neural network's resolution.
I would start with 1280 × 720 at 30 FPS, assuming your camera supports it. This gives you good spatial detail while keeping USB bandwidth and image processing manageable.
The important consideration is the size of the objects in the image. If your objects are small, a higher-resolution camera image can preserve useful detail before resizing. However, resizing to 640 × 640 cannot recover detail that the model's input resolution cannot represent.
Also, use letterboxing (preserving aspect ratio and padding) rather than stretching the image. This avoids distorting the objects.

### An important detail: your Hailo model has a fixed input size

The model's compiled HEF file determines the input dimensions. For example, if you compile a YOLO model at 640 × 640, that HEF expects that input shape. It cannot simply accept arbitrary dimensions.

This means you can change the camera resolution without changing the model, provided your preprocessing converts the image to the required dimensions.
There is also a useful option for small objects: Hailo's tiling approach can divide a high-resolution image into overlapping regions and process them separately. This can improve detection of small objects, although it costs additional inference time.

## 2. Custom model training

I would use Ultralytics YOLO for training.
The workflow would be:

1. Collect training images
   Photograph or record examples of the three object types under the lighting and viewing conditions you expect.
2. Annotate the images
   Draw bounding boxes around each instance and assign one of three class labels.
3. Train the YOLO model
   Use Python and Ultralytics on a computer with a GPU or a cloud GPU.
4. Compile for Hailo
   Convert the trained model into a Hailo Executable Format (`.hef`) using the Hailo Dataflow Compiler.
5. Deploy to Raspberry Pi
   Copy the HEF and class-label configuration to the Pi and run inference using HailoRT.

Useful references:

- Ultralytics Hailo integration https://docs.ultralytics.com/integrations/hailo/
- Hailo Model Zoo https://github.com/hailo-ai/hailo_model_zoo
- Hailo applications https://github.com/hailo-ai/hailo-apps

One hardware naming detail worth checking: Raspberry Pi's 13 TOPS AI HAT+ uses the Hailo-8L designation. The Hailo-8 is the 26 TOPS variant. This matters when compiling your model because the HEF must target the correct accelerator.

## 3. Handling an unknown number of objects

I would not impose a fixed number of tracked objects.
Instead, each frame would produce a dynamically sized collection:

```
objects = [
    {
        "id": 1,
        "class_id": 0,
        "cx": 245,
        "cy": 312,
        "confidence": 0.96
    },
```

```
    {
        "id": 2,
        "class_id": 0,
        "cx": 512,
        "cy": 187,
        "confidence": 0.91
    },
    {
        "id": 3,
        "class_id": 2,
        "cx": 690,
        "cy": 425,
        "confidence": 0.94
    }
]
```

The number of entries changes automatically with the number of detections.

I would use ByteTrack initially. It maintains tracking IDs across frames, so an object can retain its identity as it moves around the screen.

I would also make these parameters configurable:

- Detection confidence threshold.
- Minimum object size.
- Tracking association threshold.
- Maximum allowed tracking age.
- Maximum number of simultaneous tracks (optional safety limit).

# 4. NetworkTables integration

Since you're using NetworkTables, I'd make the vision application publish directly from the Raspberry Pi.

I suggest a table structure like this:

/ObjectVision

SYSTEM STATUS

- `connected` — Boolean
- `heartbeat` — Integer
- `frameTimestamp` — Double
- `fps` — Double
- `objectCount` — Integer

  OBJECT DATA
- `objectIds` — Integer array
- `classIds` — Integer array
- `centroidsX` — Double array
- `centroidsY` — Double array
- `confidences` — Double array
- `boundingBoxes` — Double array

For example, the arrays might contain:

```
objectIds  = [1, 2, 3]
```

```
classIds   = [0, 0, 2]
centroidsX = [245.0, 512.0, 690.0]
centroidsY = [312.0, 187.0, 425.0]
```

All arrays use the same ordering, so index 0 in each array describes the same object.

I would also publish a frame sequence number after updating the arrays. This gives the receiving robot code a way to determine when a complete new set of detections has been published. NetworkTables supports array publishers directly.

### My recommendation for the NetworkTables data format

For your application, I would also consider publishing one flattened `DoubleArray` containing all object records, in addition to the individual arrays.

For example, each object could occupy nine values:

```
[id, class, cx, cy, confidence, x1, y1, x2, y2]
```

This provides a single topic update containing all detections from a frame. It avoids the possibility of the robot reading arrays from different frames.

I would use:

- `/ObjectVision/objects` — flattened detection records.
- `/ObjectVision/objectCount` — number of records.
- `/ObjectVision/heartbeat` — incrementing frame counter.
- `/ObjectVision/frameTimestamp` — capture timestamp.

The robot can then reconstruct each object's information from one array.

## 5. Proposed project organization

I'd organize the Python project this way:

```
ObjectVision/
│
├── main.py
├── config.json
│
├── camera/
│   └── usb_camera.py
│
├── detection/
│   ├── hailo_detector.py
│   └── preprocessing.py
│
├── tracking/
│   └── byte_tracker.py
│
├── vision/
│   ├── centroid.py
│   └── visualization.py
│
├── networking/
│   └── networktables.py
│
├── models/
│   ├── detector.hef
│   └── labels.json
```

```
└── requirements.txt
```

This separates the camera, neural network, tracking, and communications. You can change the model without rewriting your NetworkTables interface.

# 6. A few implementation preferences

Here is what I would choose initially:

| Component | Recommendation |
|---|---|
| Language | Python |
| AI accelerator | Hailo-8L / Hailo-8, as verified |
| Detection | Custom-trained YOLO |
| Tracking | ByteTrack |
| Camera | USB UVC |
| Initial camera resolution | 1280 × 720 |
| Initial model input | 640 × 640 |
| Image preprocessing | Letterbox |
| Centroids | Original camera pixel coordinates |
| Communications | NetworkTables NT4 |
| Output | Dynamically sized object collection |

One additional consideration: I would preserve the camera capture timestamp throughout the pipeline rather than timestamping only when inference finishes. This will be useful when the robot consumes vision measurements.

# 7. One question about the coordinate system

Since you're using NetworkTables, I'm assuming the Raspberry Pi will communicate with a roboRIO running your FRC robot code.

How would you prefer the centroid coordinates to be represented?

Centroid coordinate representation

Pixel coordinates

X = 0–1279, Y = 0–719 for a 1280 × 720 image.

Normalized coordinates

X and Y range from 0.0 to 1.0, independent of resolution.

Both

Publish pixel and normalized coordinates.

NetworkTables destination

roboRIO (FRC robot)

Another NetworkTables server

Use these settings

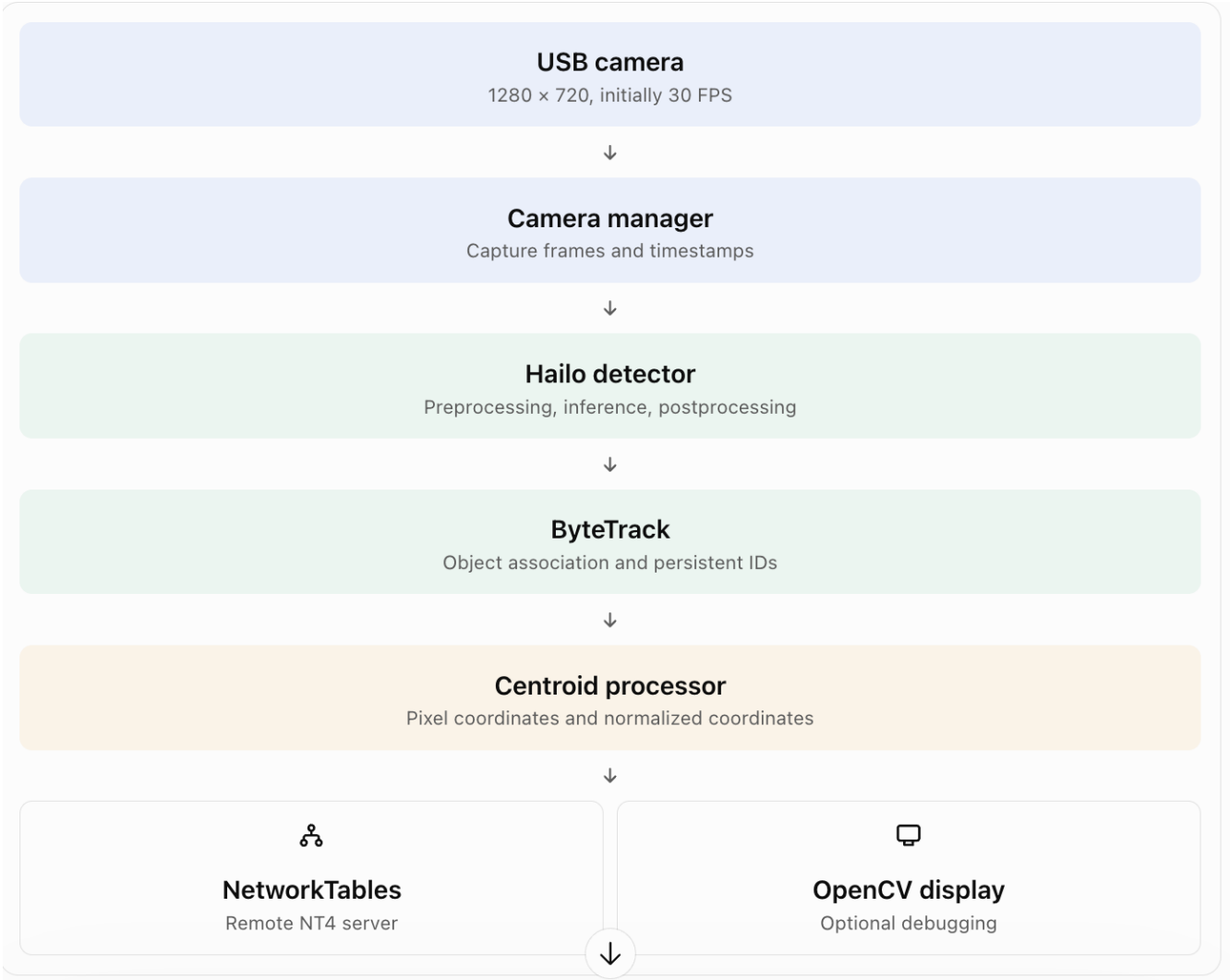
My preference is both pixel and normalized coordinates. Pixel coordinates are useful for image processing and visualization, while normalized coordinates make downstream robot code independent of camera resolution.

For the ObjectVision implementation, use both centroid coordinates. NetworkTables destination: other. Please proceed with the Python architecture.

I'll proceed with a modular Python application named ObjectVision, with NetworkTables as an independent output interface.
Since the NetworkTables destination is another server rather than the roboRIO, I'll make the server address and port configurable. The application won't assume an FRC team number or a particular robot network.
The design will support:

- Three custom-trained object classes, with class names configurable.
- Any practical number of simultaneous detections.
- Unique tracking IDs for individual objects.
- Pixel and normalized centroid coordinates.
- Hailo-accelerated inference.
- USB camera input.
- NetworkTables NT4 publishing.
- Optional live video display for debugging.

# 1. Final architecture




One implementation choice is important: I would keep Hailo inference and tracking in separate modules, so the detector can eventually be replaced without changing the tracking or NetworkTables code.

# 2. Project configuration

I want to keep hardware and application settings outside the Python source code. That way, you can change the camera, model, network server, and tracking parameters without editing the application. Here is the proposed `config.json`:

```
{
  "camera": {
    "device": "/dev/video0",
    "width": 1280,
    "height": 720,
    "fps": 30,
    "pixel_format": "MJPG"
```

```
  },
  "model": {
    "hef_path": "models/detector.hef",
    "input_width": 640,
    "input_height": 640,
    "class_names": [
      "object_a",
      "object_b",
      "object_c"
    ],
    "confidence_threshold": 0.50,
    "nms_threshold": 0.45
  },
  "tracking": {
    "enabled": true,
    "algorithm": "bytetrack",
    "track_buffer": 30,
    "max_objects": 0
  },
  "networktables": {
    "enabled": true,
    "server": "192.168.1.100",
    "port": 5810,
    "identity": "ObjectVision",
    "table": "/ObjectVision",
    "publish_rate": 30
  },
  "display": {
    "enabled": true,
    "show_fps": true,
    "show_bounding_boxes": true,
    "show_centroids": true,
    "show_tracking_ids": true
  }
}
```

A few things to note:

- `max_objects: 0` means unlimited by application configuration. Actual capacity will depend on available processing time and tracker settings.
- The three class names are placeholders.
- The server address is an example and should be replaced with your actual NT server address.
- The model input dimensions will ultimately be read from or checked against the HEF metadata.

# 3. Python modules

I'd implement the following modules first.

| Module | Responsibility |
|---|---|
| `usb_camera.py` | Camera capture and timestamps |
| `hailo_detector.py` | Hailo inference |
| `preprocessing.py` | Letterbox and coordinate transformations |
| `byte_tracker.py` | Maintain object identities |
| `centroid.py` | Calculate both coordinate systems |
| `networktables.py` | Publish detections |

| | |
|---|---|
| `visualization.py` | Draw results on camera frames |
| `main.py` | Run the processing pipeline |

### Centroid calculations

For an image with width (W) and height (H):

cx = (x1+x2) / 2

cy = (y1+y2) / 2

Normalized coordinates:

cxn = cx / W

cyn = cy / H

For example, with a 1280 × 720 image and a centroid at (640, 360):

| Coordinate | Value |
|---|---|
| Pixel X | 640 |
| Pixel Y | 360 |
| Normalized X | 0.5 |
| Normalized Y | 0.5 |

The normalized origin will be the upper-left corner, just like OpenCV's pixel coordinate system. X increases to the right and Y increases downward.

### Object data structure

I'd use a dataclass to represent each tracked object:

```
from dataclasses import dataclass

@dataclass
class TrackedObject:
    track_id: int
    class_id: int
    class_name: str
    confidence: float

    x1: float
    y1: float
    x2: float
    y2: float

    cx: float
    cy: float

    cx_norm: float
    cy_norm: float

    timestamp: float
```

This gives every downstream module a consistent data format, regardless of which model or tracking algorithm is being used.

## 4. NetworkTables publisher

I recommend using RobotPy's `ntcore` library rather than the older `pynetworktables` library.

The Raspberry Pi will operate as an NT4 client and connect to the server you specify. The default NT4 port is 5810.

The main topic structure will be:

```
/ObjectVision/
    connected
    heartbeat
    fps
    objectCount
    frameTimestamp
    objects
```

The `objects` topic will contain nine values per object:

```
[
  id, classId, cx, cy, cxNorm, cyNorm,
  confidence, x1, y1, x2, y2
]
```

That is actually eleven values per object, and I recommend keeping all eleven. The flattened format makes it possible to publish the complete detection list in one topic update.

Here is the publisher implementation:

```
import ntcore
import time

class ObjectVisionNT:
    def __init__(self, server, port=5810,
                 identity="ObjectVision",
                 table_name="/ObjectVision"):

        self.inst = ntcore.NetworkTableInstance.getDefault()
        self.inst.setServer(server, port)
        self.inst.startClient4(identity)
        self.table = self.inst.getTable(table_name)
        self.connected = self.table.getBooleanTopic(
            "connected"
        ).publish()
        self.heartbeat = self.table.getIntegerTopic(
            "heartbeat"
        ).publish()
        self.object_count = self.table.getIntegerTopic(
            "objectCount"
        ).publish()
        self.fps = self.table.getDoubleTopic(
            "fps"
        ).publish()
        self.timestamp = self.table.getDoubleTopic(
            "frameTimestamp"
        ).publish()
        self.objects = self.table.getDoubleArrayTopic(
            "objects"
        ).publish()
        self.frame_number = 0

    def publish(self, tracked_objects, fps, timestamp):
        self.frame_number += 1
        data = []
        for obj in tracked_objects:
            data.extend([
                float(obj.track_id),
                float(obj.class_id),
```

```
                obj.cx,
                obj.cy,
                obj.cx_norm,
                obj.cy_norm,
                obj.confidence,
                obj.x1,
                obj.y1,
                obj.x2,
                obj.y2
            ])
        self.connected.set(self.inst.isConnected())
        self.object_count.set(len(tracked_objects))
        self.fps.set(float(fps))
        self.timestamp.set(float(timestamp))
        self.objects.set(data)

        # Write heartbeat last to identify the completed frame.
        self.heartbeat.set(self.frame_number)

    def close(self):
        self.connected.set(False)
        self.inst.flush()
        self.inst.stopClient()
```

The heartbeat is a frame sequence counter, not a connection indicator. The `connected` topic reports whether the NT4 client is connected to its server.
One subtlety: NT4 publishes individual topics, not a transactional group of topics. The flattened array makes the object records a single update; the heartbeat is a useful completion marker for the accompanying status topics.

# 5. Hailo integration

This is the part I would implement carefully because Hailo's inference API and output formats depend on the model and installed HailoRT version.
I recommend using the current Hailo application framework as the starting point rather than writing a low-level HailoRT pipeline immediately. (https://github.com/hailo-ai/hailo-apps)

The current standalone object-detection application supports USB camera input, HEF models, and tracking. It is a useful reference for our detector integration.

On the Raspberry Pi, the installation workflow is:

```
git clone https://github.com/hailo-ai/hailo-apps.git
```

```
cd hailo-apps
```

```
./install.sh
```

For the standalone application, Hailo documents creating a virtual environment with access to the system-installed Hailo packages:

```
python3 -m venv --system-site-packages .venv
```

```
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Before integrating ObjectVision, verify the accelerator:

```
hailortcli fw-control identify
```

And inspect a model:

```
hailortcli parse-hef models/detector.hef
```

## Detector interface

I would give the detector a simple interface:

```
class HailoDetector:

    def __init__(self, config):
        # Load HEF and configure Hailo
        pass
    def detect(self, frame):
        """
        Input:
            OpenCV BGR image
        Output:
            List of detections
        Each detection:
            bbox
            class_id
            confidence
        """
        pass
    def close(self):
        pass
```

This is an interface specification, not yet a working Hailo implementation. The actual inference and postprocessing code will be adapted to the selected HEF's output format.

## Image preprocessing

The preprocessing module will perform letterboxing.

For example, a 1280 × 720 image resized into a 640 × 640 network input becomes:

## 640 × 640 model input




1280 × 720 image
The image occupies 640 × 360 pixels, with 140 pixels of padding above and below.
The preprocessing module must retain the resize scale and padding values so that the detection coordinates can be transformed back into camera coordinates.
This is particularly important for accurate centroids.

# 6. Processing loop

The main application will look conceptually like this:

```
while running:
    # Capture frame and timestamp
    frame, timestamp = camera.read()

    # Run Hailo inference
    detections = detector.detect(frame)

    # Associate detections with existing objects
    tracks = tracker.update(detections)

    # Calculate pixel and normalized centroids
    objects = centroid_processor.process(
        tracks,
        frame.shape
    )

    # Publish complete object list
    networktables.publish(
```

```
        objects,
        fps,
        timestamp
    )

    # Optional visualization
    if display_enabled:
        visualizer.draw(frame, objects)
```

I would also make sure that a frame with zero detections publishes an empty object array and an object count of zero. That prevents stale detections from remaining visible to the receiving application.
For performance, the camera capture should eventually run independently of inference, using a small latest-frame buffer. This prevents the application from building up a queue of old camera frames when inference takes longer than the camera frame interval.

# 7. Performance and reliability considerations

There are a few things I'd build into the application from the beginning.
Independent camera capture
Use a separate capture thread and a latest-frame buffer. Don't allow inference to accumulate old frames.
Dynamic tracking
Don't impose a fixed object count. ByteTrack should maintain active tracks dynamically. The optional maximum is a resource-protection setting, not a requirement.
Object disappearance
If an object disappears temporarily, retain its tracking ID for a configurable number of frames. If it reappears within that period, attempt to associate it with the existing track.
Network disconnection
Continue vision processing when NetworkTables is disconnected. NT4 will attempt to reconnect, and the application should resume publishing when connectivity returns.
Performance measurements
Publish:

- Camera FPS.
- Inference time in milliseconds.
- Total processing time.
- Number of detections.
- Number of active tracks.

These will help identify whether the bottleneck is the camera, Hailo, tracking, or networking.

# 8. One hardware correction

You mentioned a 13 TOPS Hailo-8. Just to make sure we target the correct hardware: the Raspberry Pi AI HAT+ 13 TOPS version uses the Hailo-8L, whereas the Hailo-8 is the 26 TOPS version.
The distinction matters when compiling your custom model.

# 9. Implementation roadmap

I'd develop this in five stages, testing each independently.

1. Camera and NetworkTables
   First milestone
   Capture USB camera frames, display them, and verify NetworkTables connectivity. Publish heartbeat and camera status.
2. Hailo object detection
   Load an existing YOLO HEF and display bounding boxes. Verify inference performance and coordinate transformations.
3. Multi-object tracking
   Integrate ByteTrack and verify that multiple objects receive persistent IDs.
4. Centroid and NetworkTables publishing
   Publish pixel and normalized coordinates, object classes, tracking IDs, and bounding boxes.
5. Custom model
   Train the three-class YOLO model, compile it for your Hailo hardware, and replace the test model.

# 10. Two details to settle before the first implementation

1. NetworkTables server: What software is hosting your NT4 server? For example, AdvantageScope, a separate Raspberry Pi, a desktop computer, or another system? We can support any compatible NT4 server.
2. Development environment: I suggest developing directly on the Raspberry Pi 5, using VS Code Remote SSH from your Mac. This lets us test the Hailo hardware without cross-platform complications.

My recommendation is to start with Milestone 1 and establish the camera and NetworkTables infrastructure before introducing Hailo. That gives us a working foundation that we can test independently of the neural network.

The development environment runs the Network Table server on a linux (Ubuntu 26) laptop. In production it will run on a Raspberry Pi compute module 5. Let's proceed with Milestone 1 using VS Code however, I have not used VS Code before and will need detailed instructions.

Great. We'll use a development arrangement that closely resembles your eventual production system, so we won't have to redesign ObjectVision when you move it to the Compute Module 5.

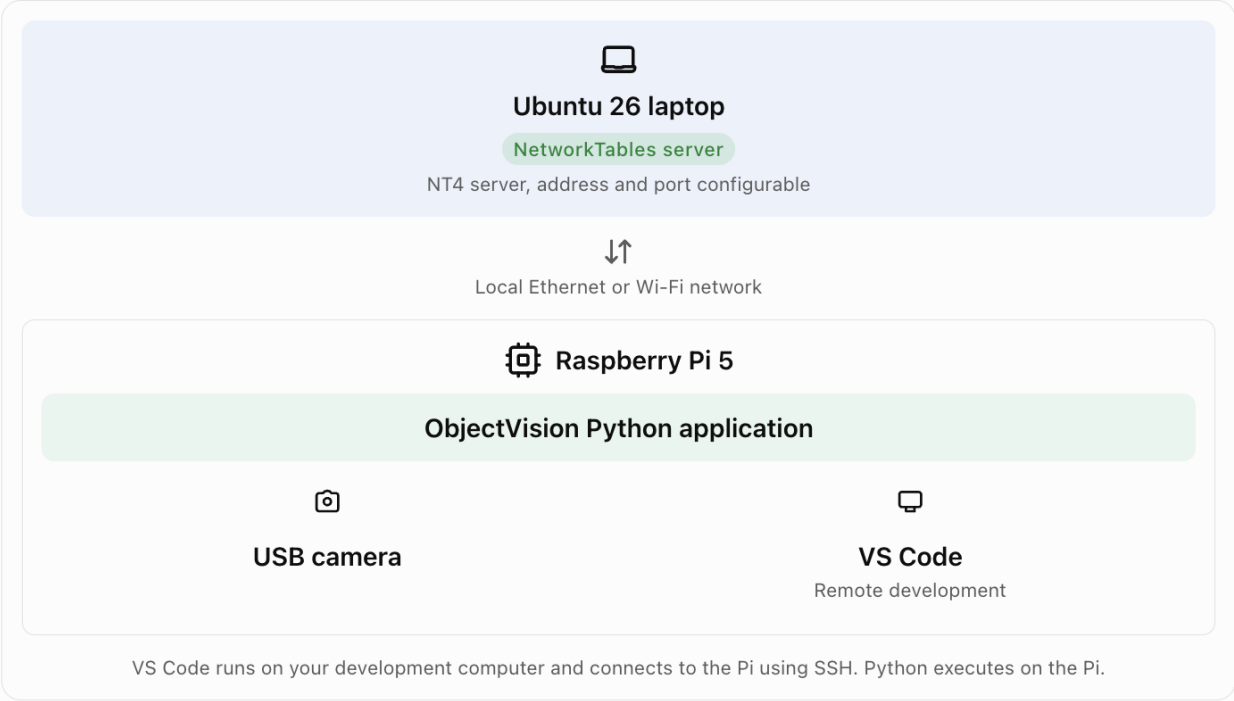
I recommend the following arrangement:

- Development computer: Your Ubuntu 26 laptop hosts the NetworkTables server.
- Vision computer: Raspberry Pi 5 runs ObjectVision and connects to the laptop over your local network.
- Development editor: Visual Studio Code (VS Code) provides the editing and debugging environment.
- Production: Move ObjectVision to the Raspberry Pi Compute Module 5 and change the NetworkTables server address in the configuration file.

We'll build and test the application incrementally. We won't introduce Hailo or object detection yet. First, we'll establish a reliable camera capture and NetworkTables connection.

# Milestone 1 — Camera and NetworkTables

## 1. Development architecture




There is one distinction worth emphasizing: VS Code is the editor, not the computer running the application. We'll edit the files remotely while the Python application executes on the Raspberry Pi. For production, we'll only need to change the NetworkTables server configuration.

## 2. Install Visual Studio Code

I'll assume you're using your Mac as the computer running VS Code, and that the Raspberry Pi 5 is running Raspberry Pi OS 64-bit. If either assumption is wrong, we can adjust the instructions.
VS Code will connect to the Pi over SSH. You won't need to install the full VS Code desktop application on the Pi.

### Step 2.1 — Install VS Code on your Mac

1. Open the official download page:
Download Visual Studio Code.
2. Download the macOS version appropriate for your Mac (Apple Silicon or Intel).
3. Open the downloaded archive.
4. Drag Visual Studio Code into your Applications folder.
5. Launch Visual Studio Code.

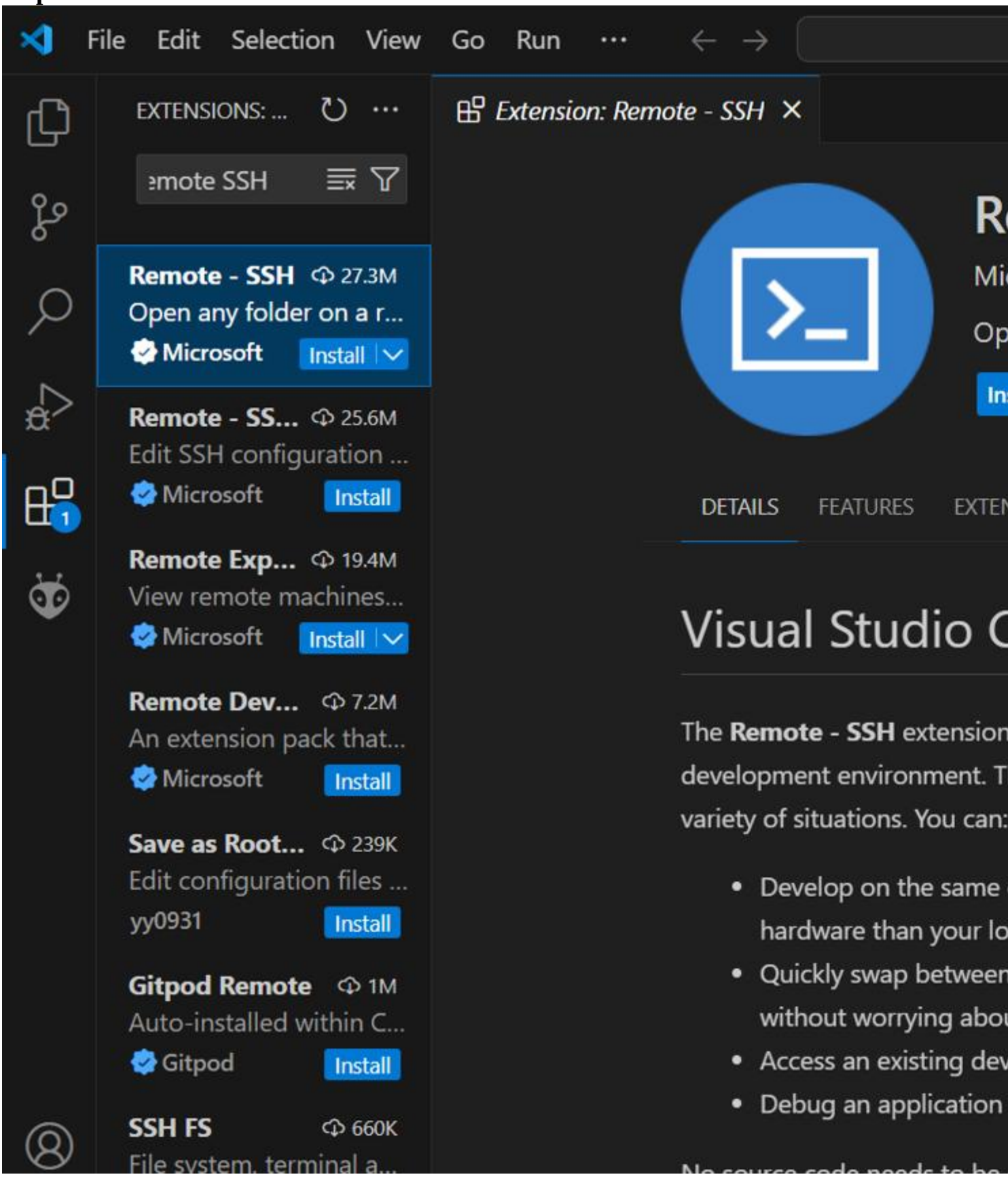
You should see an interface with:

- Explorer on the left for project files.
- Search for finding code.

- Source Control for Git.
- Run and Debug for executing Python.
- Extensions for adding capabilities.
- An editor area in the center.

Don't worry about learning all of these features now. We'll use just a few initially.

**Step 2.2 — Install the Remote SSH extension**



1. Open VS Code.
2. Click the Extensions icon on the left (four small squares).
3. Enter `Remote - SSH` in the search field.
4. Find the extension published by Microsoft.
5. Click Install.

This extension allows VS Code to edit files and execute commands on another computer over SSH. It also installs a small VS Code Server component on the remote computer.
Also install the Microsoft Python extension. We'll use it for Python syntax checking, interpreter selection, and debugging.

# 3. Prepare the Raspberry Pi 5 for remote development

Connect a monitor and keyboard to the Pi for this initial setup, or use an existing SSH connection if you already have one.

Open a terminal on the Pi.

## Step 3.1 — Update Raspberry Pi OS

Enter:

```
sudo apt update
sudo apt full-upgrade -y
```

## Step 3.2 — Install required software

```
sudo apt install -y \
    openssh-server \
    python3-venv \
    python3-pip \
    python3-opencv \
    v4l-utils \
    git
```

These packages provide:

- SSH connectivity.
- Python virtual environments.
- OpenCV.
- USB camera inspection tools.
- Git for source-code management.

## Step 3.3 — Enable SSH

```
sudo systemctl enable --now ssh
```

Verify:

```
sudo systemctl status ssh
```

You should see `active (running)`.

## Step 3.4 — Find the Pi's IP address

Enter:

```
hostname -I
```

For example, you might see:

```
192.168.1.125
```

Write down this address.

We will use it when configuring VS Code.

### Step 3.5 — Test SSH from your Mac

Open the Terminal application on your Mac.

Enter:

```
ssh YOUR_PI_USERNAME@192.168.1.125
```

Replace the username and IP address with your actual values.

The first time, SSH will ask whether you trust the host. Answer `yes`, then enter the Pi user's password.

If successful, your Mac terminal will now be connected to the Raspberry Pi.

Type:

```
hostname
```

You should see the Raspberry Pi's hostname.

Type `exit` to disconnect.

### Step 3.6 — Set up SSH key authentication

This avoids entering your Pi password every time VS Code connects.

On your Mac, enter:

```
ssh-keygen -t ed25519 -C "ObjectVision"
```

Press Enter to accept the default file location. You can optionally set a passphrase.

Next, copy your public key to the Pi:

```
cat ~/.ssh/id_ed25519.pub | \
ssh YOUR_PI_USERNAME@192.168.1.125 \
'mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys'
```

Now test:

```
ssh YOUR_PI_USERNAME@192.168.1.125
```

It should connect without asking for your Pi account password (although it may ask for your SSH key passphrase).

## 4. Connect VS Code to the Raspberry Pi

This is where the Remote SSH extension comes into play.

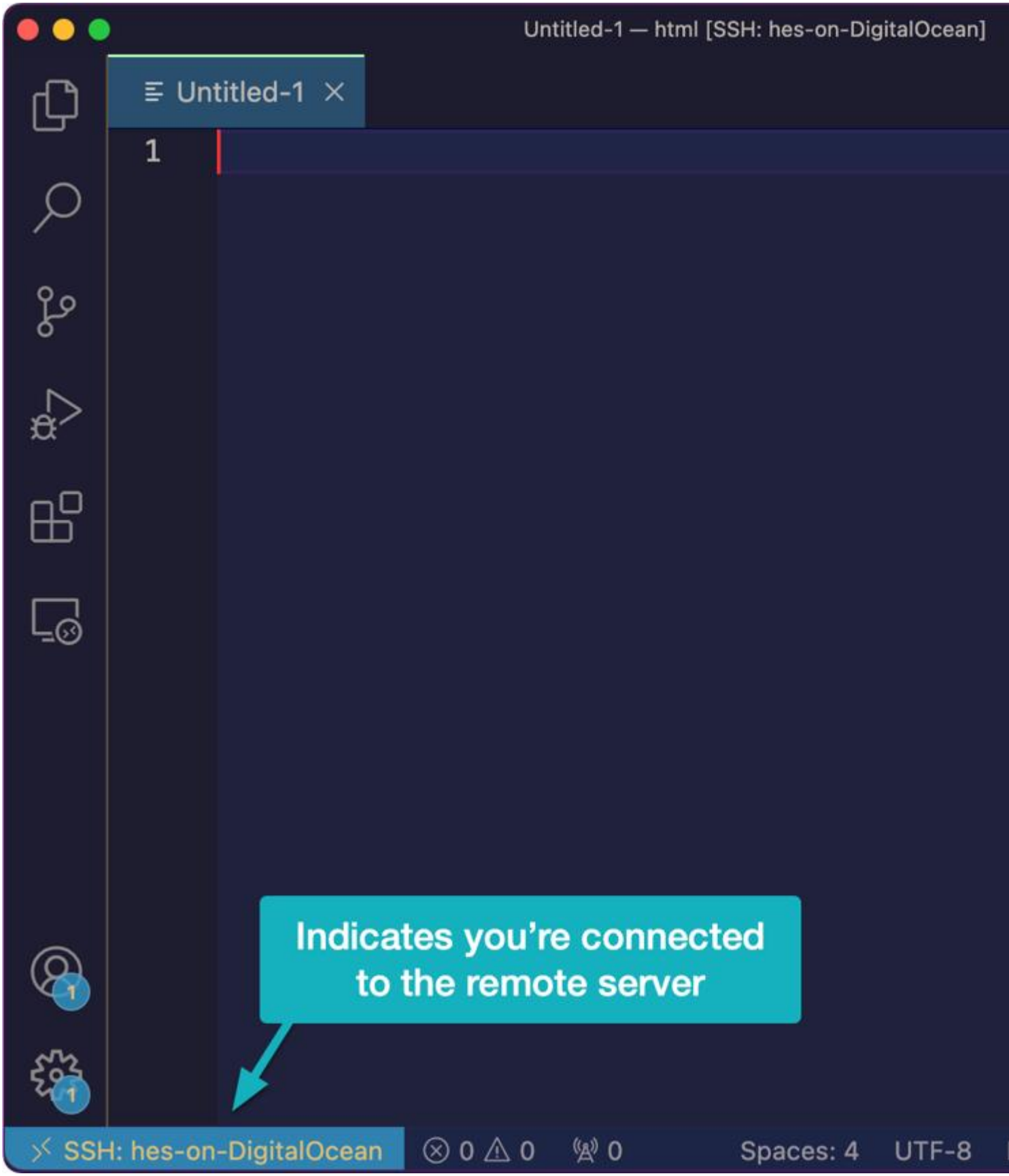
1. Open VS Code on your Mac.
2. Press `Command + Shift + P`.
   This opens the Command Palette.
3. Start typing:
   `Remote-SSH: Connect to Host...`
4. Select that command.
5. Select Add New SSH Host if you haven't configured one before.
6. Enter:

```
ssh YOUR_PI_USERNAME@192.168.1.125
```

7. Select the default SSH configuration file when prompted.
8. Press `Command + Shift + P` again.
9. Select `Remote-SSH: Connect to Host...`.
10. Select your Raspberry Pi.

VS Code will open a new window.

The first connection may take a minute because VS Code installs its remote server software.




codewithsusan.com

Look at the bottom-left corner of VS Code. You should see something similar to:

`SSH: raspberrypi`

This tells you that VS Code is operating on the Raspberry Pi.

Important: From this point forward, commands entered in the VS Code integrated terminal will execute on the Pi, not on your Mac.

The official Microsoft walkthrough is available here: Remote development using SSH.

# 5. Create the ObjectVision project

Now we'll create the project directory on the Raspberry Pi.

In VS Code, select:

Terminal → New Terminal

You should see a terminal window open at the bottom of VS Code.

Enter these commands:

```
mkdir -p ~/ObjectVision
cd ~/ObjectVision
```

```
mkdir -p camera networking logs
```

Your project will initially look like this:

```
ObjectVision/
│
├── main.py
├── config.json
│
├── camera/
│   └── usb_camera.py
│
├── networking/
│   └── nt_client.py
│
└── logs/
```

## Step 5.1 — Create a Python virtual environment

A virtual environment keeps the project's Python dependencies separate from the rest of the Pi's software.

In the VS Code terminal:

```
cd ~/ObjectVision
```

```
python3 -m venv --system-site-packages .venv
```

```
source .venv/bin/activate
```

You should now see `(.venv)` at the beginning of the terminal prompt.

Install the NetworkTables library:

```
python -m pip install --upgrade pip
```

```
python -m pip install robotpy-ntcore
```

We use `--system-site-packages` so that the virtual environment can access the OpenCV package installed through Raspberry Pi OS.

Verify both packages:

```
python -c "import cv2; print(cv2.__version__)"
```

```
python -c "import ntcore; print('NetworkTables OK')"
```

If both commands work, we're ready to create the application.

# 6. Create the configuration file

In VS Code:

1. Click File → Open Folder.
2. Enter `/home/YOUR_PI_USERNAME/ObjectVision`.
3. Click Open.
4. In the Explorer panel, click the New File icon.
5. Name the file `config.json`.

Paste this configuration:

```
{
    "camera": {
        "device": "/dev/video0",
        "width": 1280,
        "height": 720,
        "fps": 30,
        "pixel_format": "MJPG"
    },

    "networktables": {
        "server": "192.168.1.100",
        "port": 5810,
        "identity": "ObjectVision-Pi5",
        "table": "/ObjectVision"
    },

    "display": {
        "enabled": false
    }
}
```

Replace `192.168.1.100` with the Ubuntu laptop's actual IP address.

We'll keep the display disabled initially because the Pi may be running headless, without a monitor. We can add a remote video preview later.

# 7. Implement USB camera capture

Create:

`camera/usb_camera.py`

Paste the following:

```
import cv2
import time

class USBCamera:
    def __init__(self, config):
        device = config["device"]
        if isinstance(device, str) and device.isdigit():
            device = int(device)
        self.cap = cv2.VideoCapture(
            device,
            cv2.CAP_V4L2
        )
        if not self.cap.isOpened():
            raise RuntimeError(
```

```
                f"Unable to open camera: {device}"
            )
        width = config["width"]
        height = config["height"]
        fps = config["fps"]
        self.cap.set(
            cv2.CAP_PROP_FOURCC,
            cv2.VideoWriter_fourcc(
                *config["pixel_format"]
            )
        )
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
        self.cap.set(cv2.CAP_PROP_FPS, fps)
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        self.width = int(
            self.cap.get(cv2.CAP_PROP_FRAME_WIDTH)
        )
        self.height = int(
            self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
        )
        self.fps = self.cap.get(
            cv2.CAP_PROP_FPS
        )
        print("Camera initialized")
        print(f"Resolution: {self.width} x {self.height}")
        print(f"Reported FPS: {self.fps}")

    def read(self):
        success, frame = self.cap.read()
        if not success:
            return None, None
        timestamp = time.time()
        return frame, timestamp

    def close(self):
        if self.cap is not None:
            self.cap.release()
            self.cap = None
```

This module:

- Opens the USB camera.
- Requests 1280 × 720 at 30 FPS.
- Requests MJPEG format.
- Reads frames.
- Records a timestamp for each captured frame.
- Provides a clean shutdown.

The actual resolution and frame rate are reported because USB cameras don't always accept every requested setting.

# 8. Implement the NetworkTables client

Create:

`networking/nt_client.py`

Paste:

```
import ntcore

class ObjectVisionNT:

    def __init__(self, config):
        self.inst = ntcore.NetworkTableInstance.getDefault()
        identity = config["identity"]
        self.inst.startClient4(identity)
        self.inst.setServer(
            config["server"],
            config["port"]
        )

        table = self.inst.getTable(config["table"])
        self.connected = (
            table.getBooleanTopic("connected").publish()
        )
        self.camera_connected = (
            table.getBooleanTopic("cameraConnected").publish()
        )
        self.heartbeat = (
            table.getIntegerTopic("heartbeat").publish()
        )
        self.object_count = (
            table.getIntegerTopic("objectCount").publish()
        )
        self.fps = (
            table.getDoubleTopic("fps").publish()
        )
        self.timestamp = (
            table.getDoubleTopic("frameTimestamp").publish()
        )
        self.frame_width = (
            table.getIntegerTopic("frameWidth").publish()
        )
        self.frame_height = (
            table.getIntegerTopic("frameHeight").publish()
        )
        self.objects = (
            table.getDoubleArrayTopic("objects").publish()
        )
        self.frame_number = 0
        print("NetworkTables client initialized")

    def publish_frame(self, timestamp, fps, width, height):
        self.frame_number += 1
        self.connected.set(self.inst.isConnected())
        self.camera_connected.set(True)
        self.heartbeat.set(self.frame_number)
        self.object_count.set(0)
        self.fps.set(float(fps))
        self.timestamp.set(float(timestamp))
        self.frame_width.set(width)
        self.frame_height.set(height)
        # No object detections during Milestone 1.
        self.objects.set([])
```

```
    def close(self):
        self.camera_connected.set(False)
        self.connected.set(False)
        self.inst.flush()
        self.inst.stopClient()
```

For this first milestone, we're publishing zero objects because we haven't implemented detection yet. The important thing is that the NetworkTables connection is established and updated continuously. We'll extend this publisher later without changing its basic interface.

# 9. Create the main application

Create `main.py` in the project root.

```
import json
import time
import cv2

from camera.usb_camera import USBCamera
from networking.nt_client import ObjectVisionNT

def load_config():
    with open("config.json", "r") as file:
        return json.load(file)

def main():
    config = load_config()
    camera = None
    nt = None
    try:
        camera = USBCamera(config["camera"])
        nt = ObjectVisionNT(config["networktables"])
        display_enabled = config["display"]["enabled"]
        frame_count = 0
        fps = 0.0
        start_time = time.perf_counter()
        last_report = start_time
        snapshot_saved = False
        print("\nObjectVision started")
        print("Press Ctrl+C to stop\n")
        while True:
            frame, timestamp = camera.read()
            if frame is None:
                print("Camera frame capture failed")
                time.sleep(0.05)
                continue
            frame_count += 1
            now = time.perf_counter()
            elapsed = now - start_time
            if elapsed >= 1.0:
                fps = frame_count / elapsed
                frame_count = 0
                start_time = now
            height, width = frame.shape[:2]
            # Save one frame for camera verification.
            if not snapshot_saved:
                cv2.imwrite(
```

```
                    "logs/camera_test.jpg",
                    frame
                )
                snapshot_saved = True
            # Publish camera and system information.
            nt.publish_frame(
                timestamp,
                fps,
                width,
                height
            )
            # Optional local display.
            if display_enabled:
                cv2.imshow("ObjectVision", frame)
                if cv2.waitKey(1) & 0xFF == ord("q"):
                    break
            # Periodic diagnostic output.
            if now - last_report >= 2.0:
                print(
                    f"FPS: {fps:.1f} | "
                    f"Resolution: {width}x{height} | "
                    f"NT Connected: {nt.inst.isConnected()}"
                )
                last_report = now
    except KeyboardInterrupt:
        print("\nStopping ObjectVision")
    finally:
        if camera is not None:
            camera.close()
        if nt is not None:
            nt.close()
        cv2.destroyAllWindows()
        print("ObjectVision stopped")

if __name__ == "__main__":
    main()
```

Also create two empty files so Python recognizes the directories as packages. At the command line:
touch camera/__init__.py
touch networking/__init__.py

# 10. Prepare the Ubuntu laptop's NetworkTables server

There is one important distinction in our setup:

- Ubuntu laptop: NT4 server.
- Raspberry Pi: NT4 client.

The Pi must not start its own server.

On your Ubuntu laptop, open a terminal.

First, find the laptop's IP address:

```
hostname -I
```

Make sure the address matches the one you entered in `config.json`.

Next, verify that the server is listening:

```
sudo ss -ltnp | grep 5810
```

You should see a listening TCP socket on port 5810.

If Ubuntu's firewall is enabled, allow the connection:

```
sudo ufw allow 5810/tcp
```
If you already have an NT4 server running, don't start another one. Only one process can own the server port.
```
import ntcore
import time
inst = ntcore.NetworkTableInstance.getDefault()
inst.startServer()
print("NetworkTables server started on port 5810")
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    inst.stopServer()
```

The official NetworkTables API supports both starting a server and connecting a client to a specific host and port.

# 11. Test the USB camera

Before starting ObjectVision, let's verify the USB camera is recognized.

Back in your VS Code terminal on the Pi:
```
v4l2-ctl --list-devices
```
You should see your camera listed, probably as `/dev/video0`.

Next:
```
v4l2-ctl -d /dev/video0 --list-formats-ext
```
This displays the supported resolutions, frame rates, and formats.

If the camera supports 1280 × 720 MJPEG at 30 FPS, our initial settings should be appropriate.

# 12. Run ObjectVision

In the VS Code terminal:
```
cd ~/ObjectVision
source .venv/bin/activate
python main.py
```

You should see something similar to:
```
Camera initialized
Resolution: 1280 x 720
Reported FPS: 30.0

NetworkTables client initialized

ObjectVision started
Press Ctrl+C to stop

FPS: 29.8 | Resolution: 1280x720 | NT Connected: True
FPS: 30.0 | Resolution: 1280x720 | NT Connected: True
FPS: 29.9 | Resolution: 1280x720 | NT Connected: True
```

The exact frame rate will depend on the camera and its USB configuration.

The application will also save its first captured image here:
```
~/ObjectVision/logs/camera_test.jpg
```

You can open that file in VS Code's Explorer to verify that the camera is producing images.
Press `Ctrl+C` to stop.

# 13. Verify NetworkTables communication

We want to verify that the Ubuntu laptop is receiving data from the Raspberry Pi.
If you have AdvantageScope installed, you can use it as an NT4 client to inspect the topics. Connect it to the Ubuntu laptop's address.
Look for:

/ObjectVision — Expected

| Topic | Expected value |
|---|---|
| connected | true |
| cameraConnected | true |
| heartbeat | Increasing integer |
| objectCount | 0 |
| fps | Approximately 30 |
| frameWidth | 1280 |
| frameHeight | 720 |
| objects | Empty array |

If you see an increasing heartbeat, you have established the essential communication path.
A note about security: NetworkTables is intended for trusted robot networks. Don't expose port 5810 directly to the public internet.

# 14. What we've accomplished

At the end of Milestone 1, we'll have:

- A working VS Code development environment.
- Remote editing and debugging on the Raspberry Pi.
- USB camera capture.
- Configurable camera resolution and frame rate.
- Frame timestamps.
- NetworkTables NT4 client connectivity.
- Continuous heartbeat publishing.
- Camera status and performance telemetry.
- A project structure ready for Hailo integration.

We will retain this architecture when moving from the Pi 5 to the Compute Module 5.

# 15. Before we move forward

I'd like to verify three details before we start troubleshooting or adding the next features.

1. What software is running the NetworkTables server on Ubuntu 26?

Python / RobotPy ntcore

WPILib application

Another application

Not yet configured / Not sure

2. What operating system is installed on the Raspberry Pi 5?

Raspberry Pi OS 64-bit

Ubuntu

Other / Not sure

3. What USB camera are you using?

Submit environment details

My suggestion: Let's get Milestone 1 running and verified before installing or integrating the Hailo detection software. This will make debugging considerably easier.

Useful reference: Microsoft VS Code Remote SSH documentation:
https://code.visualstudio.com/docs/remote/ssh

Milestone 1 environment details:
NetworkTables server: ntcore
Raspberry Pi OS: raspios64
USB camera: Not yet selected
Please continue with the implementation.

Excellent. We now have enough information to establish the development environment.
Our configuration will be:

| Component | Selection |
|---|---|
| Development computer | Mac |
| Code editor | VS Code |
| Vision computer | Raspberry Pi 5 |
| Pi operating system | Raspberry Pi OS 64-bit |
| AI accelerator | Hailo AI HAT+ |
| NetworkTables server | RobotPy `ntcore` on Ubuntu 26 |
| NetworkTables client | Raspberry Pi 5 |
| Camera | To be selected |
| Camera interface | USB / UVC |

Not having selected a camera is perfectly fine. We'll make the camera module independent of the camera model, so you can use almost any standards-compliant USB camera.
One change I'd make to our earlier plan: Let's get the NetworkTables server running and verified on Ubuntu before connecting the Pi. This gives us two independently testable systems and makes troubleshooting much easier.

# Milestone 1A — Establish the NetworkTables server

We'll first create a small Python server on your Ubuntu laptop.

## 1.1 Check whether the Ubuntu server is already running

On the Ubuntu laptop, open a Terminal window.
Run:

```
python3 --version
```

Then:

```
sudo ss -ltnp | grep 5810
```

There are two possible outcomes:

- You see a process listening on port 5810: An NT4 server is already running. Don't start another one.
- Nothing is displayed: We'll create and start our own server.

The default NT4 port is TCP 5810. RobotPy's `ntcore` API supports both server and client operation.

## 1.2 Create the Ubuntu NT4 server (if needed)

On Ubuntu:

```
mkdir -p ~/ObjectVisionServer
```

```
cd ~/ObjectVisionServer
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install robotpy-ntcore
```
Create a file named `nt_server.py`.

```
import ntcore
import time
def get_method(obj, old_name, new_name):
    """Support both RobotPy API naming conventions."""
    return getattr(obj, old_name, None) or getattr(obj, new_name)

def main():
    inst = ntcore.NetworkTableInstance.getDefault()
    # Start the NT4 server on the default port (5810).
    get_method(inst, "startServer", "start_server")()
    print("NetworkTables server started")
    print("NT4 port: 5810")
    print("Waiting for clients...")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\nStopping server")
    finally:
        get_method(inst, "stopServer", "stop_server")()

if __name__ == "__main__":
    main()
```

Run it:
`python nt_server.py`
Leave this terminal open while testing.
This server listens on the default NT4 port. The code supports both the older camelCase and newer snake_case RobotPy method names.

# 1.3 Configure Ubuntu's firewall

In another Ubuntu terminal:
`sudo ufw status`
If the firewall is active:
`sudo ufw allow 5810/tcp`
Keep this port accessible only on your trusted local network.

# 1.4 Find the Ubuntu laptop's IP address

Run:
`hostname -I`
Suppose the result is:
`192.168.1.100`
We'll use that address in the Raspberry Pi configuration.

# Milestone 1B — Test the Raspberry Pi as an NT4 client

Before adding the camera, let's prove that the Pi can communicate with the Ubuntu server.

On the Pi, open your ObjectVision project in VS Code.

In the VS Code terminal:

```
cd ~/ObjectVision
```

```
source .venv/bin/activate
```

First verify:

```
python -c "import ntcore; print('ntcore installed successfully')"
```

If it isn't installed:

```
python -m pip install robotpy-ntcore
```

## 2.1 Create a NetworkTables test program

Create:

```
networking/nt_smoke_test.py
```

```
import ntcore
import time

def get_method(obj, old_name, new_name):
    """Support both RobotPy API naming conventions."""
    return getattr(obj, old_name, None) or getattr(obj, new_name)

def main():
    inst = ntcore.NetworkTableInstance.getDefault()
    # Start the NT4 server on the default port (5810).
    get_method(inst, "startServer", "start_server")()
    print("NetworkTables server started")
    print("NT4 port: 5810")
    print("Waiting for clients...")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\nStopping server")
    finally:
        get_method(inst, "stopServer", "stop_server")()

if __name__ == "__main__":
    main()
```

Run it using your actual Ubuntu IP address:

```
python networking/nt_smoke_test.py 192.168.1.100
```

Expected result

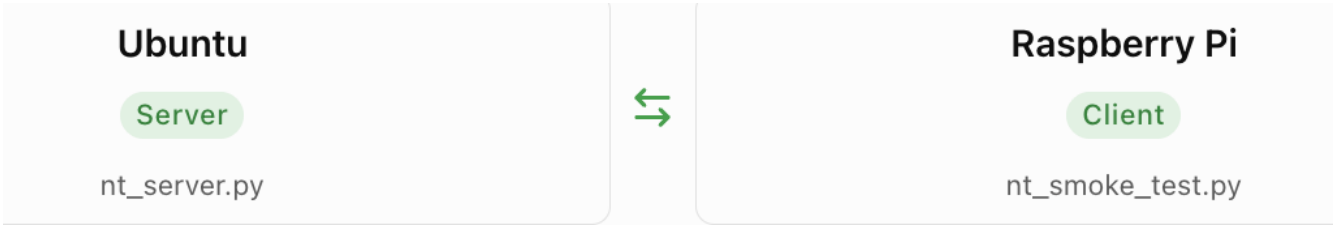
```
Connecting to NT4 server at 192.168.1.100:5810

Heartbeat: 1 | Connected: True
Heartbeat: 2 | Connected: True
Heartbeat: 3 | Connected: True
Heartbeat: 4 | Connected: True
```

If `Connected` remains false, we'll troubleshoot the network connection before proceeding.

# 3. Verify the connection

We now have two independent programs:




One compatibility adjustment before running the client: if your installed RobotPy version uses the newer snake_case API, replace the topic creation section with the following:

```
integer_topic = get_method(
    table, "getIntegerTopic", "get_integer_topic"
)

double_topic = get_method(
    table, "getDoubleTopic", "get_double_topic"
)

heartbeat = integer_topic("heartbeat").publish()
connected = table.getBooleanTopic("connected").publish()
camera_connected = table.getBooleanTopic("cameraConnected").publish()
object_count = integer_topic("objectCount").publish()
frame_width = integer_topic("frameWidth").publish()
frame_height = integer_topic("frameHeight").publish()
fps = double_topic("fps").publish()
```

For complete compatibility with the newer API, use `get_method(inst, "isConnected", "is_connected")()` when checking the connection. The Boolean topic getters have the equivalent `get_boolean_topic` name.

# 4. What I want to verify before proceeding

Please work through these steps in order:

- Check whether Ubuntu already has a process listening on port 5810.
- Start the NT4 server if one isn't already running.
- Find the Ubuntu laptop's IP address.
- Connect the Pi using the smoke-test program.
- Confirm that the heartbeat increments and `Connected` becomes `True`.

If you encounter an error, copy the complete error message from the VS Code terminal rather than trying to fix it yourself.

# 5. Next step: USB camera selection

We don't need to purchase a camera immediately, but when we get to camera testing, I suggest looking for these characteristics:

| Feature | Initial preference |
|---|---|
| Interface | USB 3.0 |
| Sensor | CMOS |
| Resolution | 1080p |
| Frame rate | 30–60 FPS |
| Focus | Fixed focus |
| Driver | Standard UVC |
| Format | MJPEG or YUYV |

Global shutter is worth considering if objects move quickly. Otherwise, a good UVC rolling-shutter camera is perfectly adequate for our initial development.

Your next checkpoint

Please send me these three results:

1. The Ubuntu laptop's IP address.
2. Whether `ss -ltnp | grep 5810` showed an existing server.
3. The output from running `nt_smoke_test.py` on the Pi.

Once we verify those results, we'll finish the camera-independent parts of Milestone 1, implement camera discovery and capture, and then move on to Hailo inference.

Official reference: RobotPy NetworkTables API documentation: https://robotpy.readthedocs.io/projects/pyntcore/en/latest/ntcore/NetworkTableInstance.html